Calculadora de IMC y Diagnóstico de Salud

Tarea Individual | Parejas — Desarrollo de Aplicaciones Móviles · React Native + TypeScript

Campo	Detalle
Asignatura	Desarrollo de Aplicaciones Móviles
Carrera	Ingeniería en Sistemas
Tipo	Trabajo en casa
Valor	10 puntos
Herramienta	React Native + TypeScript · Expo Go

1. Descripción de la Actividad

A partir del ejercicio desarrollado en clase (MisGastos), el estudiante construirá de forma autónoma una aplicación móvil en React Native con TypeScript que permita calcular el Índice de Masa Corporal (IMC) de un paciente, mostrar su diagnóstico de salud y mantener un historial de registros con **estadísticas** básicas.

El objetivo es demostrar comprensión de los conceptos vistos en clase: interfaces TypeScript, formularios controlados con useState, validación de datos, procesamiento de arreglos y estilos con StyleSheet.

Conexión con la clase

Esta tarea aplica exactamente los mismos patrones del ejercicio MisGastos:

- Union type → para los 4 diagnósticos de salud
- Interface → para el objeto Registro (nombre, peso, altura, IMC, diagnóstico)
- useState + formulario controlado → captura de datos del paciente
- Función de cálculo → fórmula del IMC separada de la validación
- Arreglo en estado → historial de registros con .map(), .filter(), .reduce()
- Estilos dinámicos → color del diagnóstico según el resultado

2. Contexto y Fórmula

2.1 Fórmula del IMC

$$IMC = peso\ (kg) \ / \ altura^2\ (m)$$

Ejemplo: peso = 70 kg · altura = 1.70 m → IMC = 70 / (1.70²) = 24.22

2.2 Tabla de Diagnósticos

Rango IMC	Diagnóstico	Color sugerido
IMC < 18.5	Bajo peso	Azul #3B82F6
18.5 – 24.9	Peso normal	Verde #10B981

25.0 – 29.9	Sobrepeso	Amarillo #F59E0B
IMC ≥ 30	Obesidad	Rojo #EF4444

Nota: Los colores son sugeridos. El estudiante puede usar otros siempre que cada diagnóstico tenga un color visualmente diferente.

3. Requerimientos Funcionales

La aplicación debe cumplir con los siguientes módulos:

3.1 Formulario de captura

- Campo de texto: Nombre del paciente
- Campo numérico: Peso en kilogramos (ej: 70.5)
- Campo numérico: Altura en metros (ej: 1.70)
- Botón Calcular IMC

3.2 Resultado del cálculo

- Mostrar el nombre del paciente en el resultado
- Mostrar el valor del IMC con exactamente 2 decimales
- Mostrar el diagnóstico correspondiente según el rango
- El diagnóstico debe tener un color diferente para cada categoría
- Botón Agregar al historial para guardar el registro en la lista

3.3 Estadísticas

- Total de registros en el historial
- IMC promedio de todos los registros
- IMC más alto registrado

3.4 Historial de registros

Igual que en el ejercicio MisGastos, la app debe mantener una lista de registros en memoria:

- Mostrar cada registro con: nombre, IMC, diagnóstico y color correspondiente
- Botón para eliminar un registro individual
- Botón para limpiar todo el historial

3.5 Validaciones obligatorias

Todas las validaciones deben mostrarse con Alert

- Los tres campos son obligatorios (no pueden estar vacíos)
- El peso debe ser un número mayor a 0 y menor a 500
- La altura debe ser un número mayor a 0 y menor a 3
- No se puede agregar al historial si el formulario no ha sido calculado primero

4. Requerimientos Técnicos

Los siguientes elementos de código son obligatorios. Su presencia se verificará en el repositorio GitHub:

Elemento	Requerimiento mínimo
interface TypeScript	Definir interface Registro con todos los campos del historial
Union type	type Diagnostico con los 4 valores posibles
useState tipado	Mínimo 4 useState con su tipo explícito
Formulario controlado	value + onChangeText en cada TextInput
StyleSheet.create()	Todos los estilos declarados en StyleSheet, sin estilos inline
Estilo dinámico	Color del diagnóstico calculado en tiempo de ejecución
Función de cálculo	calcularIMC() separada de la validación y del render
.map()	Para renderizar la lista del historial
.filter()	Para eliminar un registro por ID
.reduce()	Para calcular el IMC promedio o el total
GitHub	Repositorio público con mínimo 3 commits descriptivos

5. Instrucciones de Entrega

5.1 Documento PDF

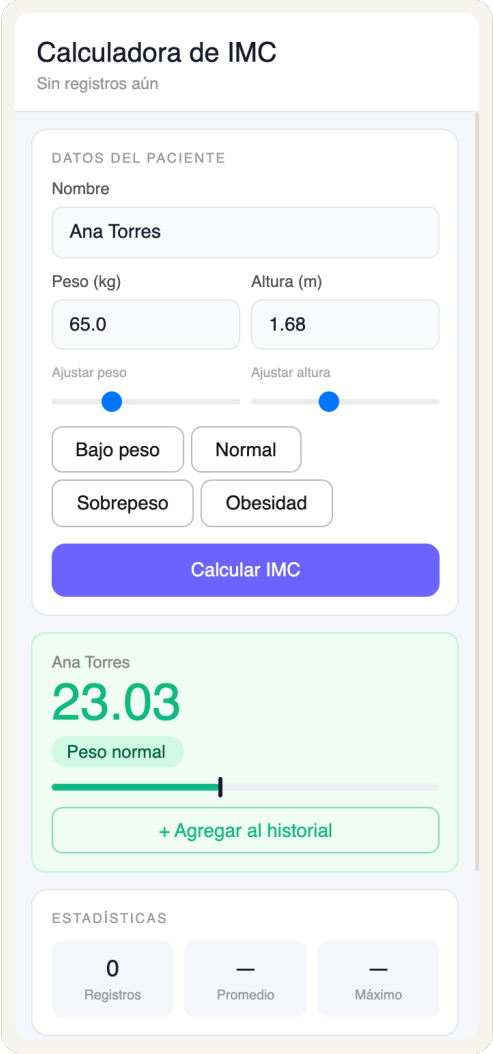
El estudiante debe entregar un único documento PDF que contenga:

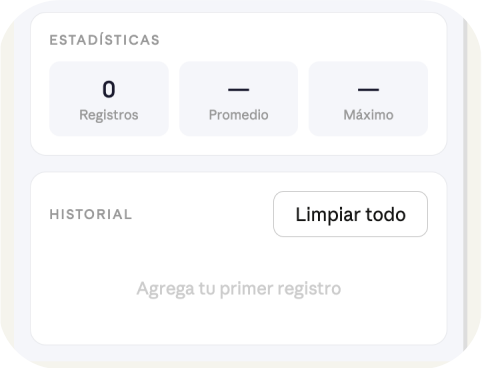
#	Captura requerida	Qué debe mostrar
1	Pantalla vacía	Estado inicial de la app, formulario en blanco, historial vacío
2	Resultado calculado	IMC calculado con diagnóstico visible y su color correspondiente
3	Historial con registros	Lista con mínimo 3 registros ingresados y estadísticas visibles
4	Mensaje de error	Alert de validación activo en pantalla (campo vacío o valor inválido)

Cómo incluir el enlace de GitHub en el PDF

- En la última página del PDF escribir: Repositorio GitHub: https://github.com/usuario/repositorio
- El repositorio debe ser público y contener el código fuente completo
- Verificar que el enlace funcione antes de entregar

5.2 UI


Calculadora de IMC
Sin registros aún
DATOS DEL PACIENTE
Nombre
Ana Torres
Peso (kg)
65.0
Altura (m)
1.68
Ajustar peso
Ajustar altura
Bajo peso
Normal
Sobrepeso
Obesidad
Calcular IMC
Ana Torres
23.03
Peso normal
+ Agregar al historial
ESTADÍSTICAS
0
Registros
—
Promedio
—
Máximo


ESTADÍSTICAS
0
Registros
—
Promedio
—
Máximo
HISTORIAL
Limpiar todo
Agrega tu primer registro

6. Rúbrica Analítica

La actividad se califica sobre 10 puntos, distribuidos en 4 criterios con igual peso.

CRITERIO 1 — Funcionalidad de la Aplicación · 2.5 puntos

Nivel	Descripción	Puntos
Excelente	Todos los módulos funcionan correctamente: cálculo del IMC, diagnóstico con color, historial completo (agregar, eliminar, limpiar todo), estadísticas y todas las validaciones con Alert.	2.5
Satisfactorio	El cálculo del IMC y el diagnóstico funcionan. El historial funciona pero le faltan una o dos funciones menores (ej: falta limpiar todo o falta una estadística).	1.75
En desarrollo	El cálculo del IMC funciona pero el historial está incompleto o las validaciones no cubren todos los casos solicitados.	1.0
Insuficiente	La aplicación no calcula correctamente el IMC o no muestra el diagnóstico. Errores que impiden el uso básico de la app.	0.5

CRITERIO 2 — Calidad del Código TypeScript · 2.5 puntos

Nivel	Descripción	Puntos
Excelente	Usa interface para Registro y union type para Diagnostico. Todos los useState tienen tipo explícito. calcularIMC() está separada del render. Sin uso de 'any'. Sin estilos inline.	2.5
Satisfactorio	Usa interface y union type correctamente. La mayoría de useState están tipados. La función de cálculo existe pero tiene algo de lógica mezclada con la validación.	1.75
En desarrollo	Define la interface o el union type pero no ambos. Algunos useState sin tipo. Estilos inline presentes junto con StyleSheet.	1.0
Insuficiente	No usa interface ni union type. No hay tipado explícito en los useState. El código no aplica los conceptos de TypeScript vistos en clase.	0.5

CRITERIO 3 — Estilos e Interfaz de Usuario · 2.5 puntos

Nivel	Descripción	Puntos
Excelente	Todos los estilos están en StyleSheet.create(). Los 4 diagnósticos tienen colores distintos aplicados dinámicamente. La interfaz es clara, organizada y usable. Los textos son legibles.	2.5
Satisfactorio	La mayoría de estilos están en StyleSheet. Los diagnósticos se diferencian visualmente pero el estilo dinámico no está implementado con array de estilos.	1.75

En desarrollo	Hay una mezcla de StyleSheet y estilos inline. Los diagnósticos no tienen diferenciación visual por color o todos tienen el mismo color.	1.0
Insuficiente	No usa StyleSheet.create(). La interfaz es difícil de usar o los elementos importantes no son visibles. No hay diferenciación visual entre diagnósticos.	0.5

CRITERIO 4 — Entregables y GitHub · 2.5 puntos

Nivel	Descripción	Puntos
Excelente	PDF entregado con las 4 capturas solicitadas y claras. Enlace de GitHub funcional con repositorio público. Mínimo 3 commits con mensajes descriptivos. Nombre del archivo correcto.	2.5
Satisfactorio	PDF entregado con 3 de las 4 capturas. GitHub funcional pero con menos de 3 commits o mensajes genéricos (ej: 'primer commit', 'cambios').	1.75
En desarrollo	PDF entregado con solo 1 o 2 capturas. El repositorio existe pero es privado o el enlace no funciona al momento de la revisión.	1.0
Insuficiente	No se entrega PDF o no se incluye enlace de GitHub. Las capturas no corresponden a la aplicación desarrollada.	0.5

Resumen de Calificación

Criterio	Ponderación	Excelente	Satisfactorio	En desarrollo	Insuficiente
1. Funcionalidad de la app	2.5 pts	2.5	1.75	1.0	0.5
2. Calidad del código TypeScript	2.5 pts	2.5	1.75	1.0	0.5
3. Estilos e interfaz de usuario	2.5 pts	2.5	1.75	1.0	0.5
4. Entregables y GitHub	2.5 pts	2.5	1.75	1.0	0.5
TOTAL	10 pts	10	7	4	2

7. Recomendaciones

Consejos para completar la tarea con éxito

- Empieza por types/index.ts: define la interface Registro y el union type Diagnostico antes de escribir cualquier componente.

- Usa el código de MisGastos como referencia directa: la lógica de agregar, eliminar y calcular estadísticas es la misma.
- Toma las capturas de pantalla en un celular real con Expo Go, no desde el emulador, para mejor calidad visual.
- Haz al menos un commit por cada módulo terminado: uno al terminar el formulario, otro al terminar el historial, otro al terminar las estadísticas.
- Verifica que tu repositorio GitHub sea público antes de entregar. Un repositorio privado no puede ser revisado.
- Prueba todas las validaciones antes de tomar las capturas: campos vacíos, letras en campo numérico, valores fuera de rango.

Errores frecuentes que debes evitar

- No uses gastos.push() ni modifiques el arreglo directamente: usa setRegistros([nuevo, ...registros]).
- No olvides la prop key en el .map() del historial: debe ser el id único de cada registro.
- No dejes ningún useState sin tipo explícito: escribe siempre useState<string>('') no useState('').
- No uses estilos inline (style={{ color: 'red' }} dentro del JSX): todo debe estar en StyleSheet.create().